Отчёт по лабораторной работе №4

Дисциплина: «Операционные системы, среды и оболочки».

Тема: взаимодействие со стандартными приложениями Windows на VBScript/JScript.

Студентка: _____. Группа: _____.

Преподаватель: _____. Дата сдачи: _____.

Цель работы

Изучить запуск приложений и передачу им клавиатурных команд через WScript.Shell, взаимодействие с файловой системой через Scripting.FileSystemObject и получение пользовательского выбора через Popup. Реализовать три задания из приложенного PDF с учётом Windows 11.

Исходные требования и соответствие файлов

Задание 1: script1.vbs запускает вычисление 9*6 через Калькулятор и вывод в Блокнот. Задание 2: script2.vbs вычисляет F7+7F, получает HEX и DEC, формирует поясняющие строки и возвращает обычный режим. Задание 3: script3.js читает text1.txt построчно, поддерживает основания 16/10/8/2, дроби, четыре операции и скобки, спрашивает основание результата и записывает text2.txt. Размещение C:\STUDENT\группа\Lab4 выполняется пользователем при распаковке на Windows; облачная копия находится в /workspace/alo/Lab4.

Теоретические примеры Internet Explorer, Run, Exec и перечни методов в PDF не сформулированы как отдельные практические задания. Internet Explorer не используется, так как для трёх заданий он не нужен и в Windows 11 устарел.

Среда и метод проверки

Подготовка выполнена в облачном Linux без доступа к ноутбуку. Доступен Node.js для тестирования общей логики. Windows Script Host, Windows PowerShell и графические приложения Windows отсутствуют. Это позволяет проверить парсер и преобразования, но не позволяет подтвердить успешную передачу клавиш окнам Windows 11. Непроверенные части явно отделены от выполненных проверок.

Этап 1. Архитектура

Созданы три требуемых файла и общие модули. VBScript и JScript используют FileSystemObject для файлов и WScript.Shell для запуска. Калькулятором управляет calculator.ps1: он создаёт тот же COM-объект WScript.Shell, активирует окно по PID, вызывает SendKeys, делает задержки 500 мс и получает результат через Ctrl+C и буфер обмена .NET. PowerShell выбран как мост к буферу обмена Windows 11; вычисления не заменяются

арифметикой PowerShell.

Блокнот получает заранее записанный Unicode-файл. Это сохраняет русский текст без зависимости от текущей раскладки. Приложения запускаются в пользовательском сеансе без повышения прав. Буфер обмена заменяется, поэтому перед запуском пользователь сохраняет его важное содержимое.

Этап 2. Первое задание

script1.vbs загружает common.vbs, передаёт мосту операнды 9 и 6 и операцию *. Мост запускает Калькулятор, выбирает обычный режим Alt+1, очищает состояние Escape, вводит числа и операцию и копирует результат. Сценарий проверяет полученное значение 54, создаёт results/script1-result.txt и открывает его в Блокноте.

Проверочное вычисление: 9 × 6 = 54. В коде нет выдачи 54 при отказе Калькулятора: контрольная константа используется только для проверки уже полученного ответа. Оконный запуск на Windows в облаке не выполнялся.

Этап 3. Второе задание

Выбирается «Программист» (Alt+3), HEX (F5), очищается текущее значение и вводится F7+7F=. Результат копируется, затем выбирается DEC (F6), снова копируется результат, после чего включается «Обычный» (Alt+1). В Блокнот записываются две поясняющие строки в точном формате PDF.

Проверка: $F7_{16} = 15 \times 16 + 7 = 247_{10}$; $7F_{16} = 7 \times 16 + 15 = 127_{10}$; сумма $= 374_{10} = 176_{16}$. Ожидаемый вывод: «Шестнадцатеричный результат: 176» и «Десятичный результат: 374». При других скопированных значениях выдаётся ошибка. Реальная работа горячих клавиш требует проверки установленной версии Windows Calculator.

Этап 4. Лексический и синтаксический разбор

engine.js реализует рекурсивный спуск. Правила: expression = term ((+|-) term)*; term = atom ((*|/) atom)*; atom = число | (+|-) atom | (expression). Поэтому умножение и деление имеют приоритет над сложением и вычитанием; операции одного уровня выполняются слева направо; скобки изменяют порядок.

Префиксы определяют основание числа: 0x — 16, @ — 2, ведущий ноль перед цифрой — 8, иначе — 10. Точка разделяет целую и дробную части. При переводе цифры целой части накапливаются по формуле n = n × b + d; дробной — как сумма d/b^k. Каждая цифра проверяется на допустимость для основания.

Пример: 0xA.8 = 10 + 8/16 = 10.5; @1.1 = 1 + 1/2 = 1.5; сумма 12. Для выражения

(0x10+010)*@10 сначала вычисляется 16+8, затем 24*2, результат 48.

Входные выражения не передаются eval. Единственный eval в script3.js загружает поставляемый локальный engine.js, поскольку у WSH нет CommonJS require. Произвольные команды из text1.txt не исполняются.

Этап 5. Арифметика через Калькулятор

Дерево обходится в порядке зависимостей. Каждый бинарный узел вызывает мост с двумя десятичными операндами и знаком операции. Например, для 2+3*4 выполняются запросы 3*4 и 2+12. Мост получает каждый ответ из самого Калькулятора. Для одиночного числа и унарного выражения без бинарных операций также выполняется ввод значения и чтение результата в Калькуляторе.

Преобразование оснований и обработка знака выполняются JScript. Дроби не отправляются в режим «Программист», который в Windows не поддерживает дробные вычисления: используется обычный режим. Так арифметика остаётся в приложении, а особенности формата входа и вывода реализуются сценарием.

Этап 6. Диалоги и файлы

С помощью Popup с кнопками Да/Нет/Отмена задаётся одно из четырёх оснований результата. Последовательность диалогов описана в инструкции Windows11. input читается построчно через OpenTextFile/ReadLine/AtEndOfStream. Результаты накапливаются в порядке обработки и записываются CreateTextFile/WriteLine в text2.txt. Все открытые потоки закрываются Close.

Ошибки отдельной строки записываются с номером строки и не мешают обработке последующих выражений. Отмена диалога не изменяет предыдущий выходной файл. Код выхода 0 означает отсутствие ошибок обработки, 1 — хотя бы одну ошибку. Запись результата в Блокнот и text2.txt использует UTF-16LE; входной файл содержит ASCII.

Этап 7. Точность и ограничения

Числа JScript представлены IEEE-754 double. Это учебная реализация, а не произвольная точность: целые значения ограничены модулем 9007199254740991; дроби могут иметь погрешность округления. После двенадцати дробных цифр представление усекается, при ненулевом остатке добавляется [truncated]. Пример: 1/3 в двоичной системе имеет бесконечную запись.

Ввод в Калькулятор разворачивается из экспоненциальной записи в десятичную, не более 31 знака после точки; ненулевые значения, округляющиеся до нуля, вызывают ошибку. Точность самого Калькулятора может быть выше, но при чтении результата в JScript она ограничивается double. Экспоненциальная запись во входном файле не поддерживается.

Длина выражения ограничена 2000 символами, вложенность — 64 уровнями; это защищает от переполнения стека.

Автоматизация зависит от стандартного Калькулятора Microsoft, его горячих клавиш, английской раскладки, возможности активации окна, регионального десятичного разделителя и отсутствия действий пользователя. Она намеренно прекращает вычисление при невозможности прочитать ответ. Эти ограничения требуют проверки на конкретной сборке Windows 11 и не скрываются тестовой арифметикой.

Этап 8. Выполненные проверки

Реально выполнено: node tests/test-engine.js — 48 успешных проверок; журнал сохранён в results/cloud-validation.txt. Проверены все 12 примеров text1.txt, дополнительные приоритеты/ассоциативность, отрицательные числа, скобки, дроби, четыре основания, неверные цифры, отсутствующие операнды, лишние символы, деление на ноль, слишком большие числа и чрезмерная вложенность. Проверяется порядок обращений к арифметическому callback: для 2+3*4 сначала *, затем +.

В этих тестах callback использует арифметику Node.js, чтобы независимо проверить разбор и преобразования. Эти 48 проверок не запускают Windows Calculator. Файл expected-results.json содержит рассчитанные эталоны по каждому основанию, а не наблюдения работы Windows.

Дополнительно node --check проверяет синтаксис engine.js, script3.js и тестов. VBScript, PowerShell и сквозные сценарии Windows в этом Linux не исполнялись; их работоспособность не объявляется доказанной. Подробный ручной сценарий проверки и устранения ошибок дан в Windows11.md.

Этап 9. Контрольные результаты

Для десятичного вывода последовательность примеров даёт: 9*6 → 54; 0xF7+0x7F → 374; (0x10+010)*@10 → 48; 0xA.8+@1.1 → 12; 010.4/2 → 4.25; 2+3*4 → 14; (2+3)*4 → 20; -5+2 → -3; 7/2 → 3.5; 0xFF → 255; @101.01 → 5.25; 0.5+0.25 → 0.75.

Для 374: HEX 0x176, DEC 374, OCT 0566, BIN @101110110. Во втором задании префикс 0x не добавляется, чтобы соответствовать формату строки из PDF.

Этап 10. Места для фактических доказательств Windows

1. Вставить скриншот Калькулятора с 54 и Блокнота после script1.vbs.
2. Вставить скриншот двух подписанных результатов script2.vbs и обычного режима Калькулятора.
3. Вставить скриншот выбора основания, text1.txt и полученного text2.txt.
4. Записать свою версию Windows, версию Калькулятора, дату запуска и фактические коды

завершения.

Эти пункты оставлены для реально выполненного локального запуска. Облачная среда не предоставляет Windows GUI, поэтому подлинные скриншоты и подтверждение GUI-проверок здесь получить невозможно.

Вывод

Подготовлены исходные файлы всех трёх заданий, входные примеры, эталонные ответы, воспроизводимые тесты, инструкция Windows 11 и комплект отчёта. Ядро разбора и перевода проверено 48 автоматическими проверками. Для окончательной приёмки требуется сквозной запуск сценариев на Windows 11 по приложенной инструкции; отсутствие этой проверки является ограничением облачной среды.